REPUBLIC OF THE PHILIPPINES
BICOL UNIVERSITY
POLANGUI
COMPUTER STUDIES DEPARTMENT
Polangui, Albay

ISO 9001: 2015
SOCOTEC SCP000722Q

BAGONG PILIPINAS

WEB SYSTEMS AND TECHNOLOGIES

Phase 4 – Form Submission and Data Insertion

LAB REPORT

Uniformity: An E-Commerce platform for Pre-Loved School and Professional Uniforms

GROUP NAME: TechNic **Course & Block:** BSIT-2A		
MEMBERS	**ROLE**	**SIGNATURE**
Cipriano, Mcxyron B.	*Back-End Developer*	
Hernandez, Kurt Jushua S.	*Database Manager/Tester & Debugger*	
Registrado, Mharie Franz	*Tester & Debugger*	
Red, Paul Orlando B.	*Project, Database, GitHub Manager & Documentation Officer*	
Romano, Jay L.	*Front-End Developer*	

Submitted to:

Guillermo V. Red, Jr., DIT

Assistant Professor IV

I. Overview

This lab report documents Phase 4 of our final web project, which is Form Submission and Data Insertion. Building on the static frontend from Phase 3 and the fully tested REST API backend from Phase 2, In this Phase 4, we now connect the HTML forms to the live MongoDB database using the JavaScript Fetch API.

The primary deliverable of this phase is **api.js** — it is a dedicated JavaScript module inside **frontend/js/** that handles all form submissions, client-side validation, loading states, and user feedback (success/error messages) without reloading the page.

Note: *The toast notifications, button loading spinners, and inline field errors were added as part of the requirement to "display success/error messages to users based on the API response."* ***These are UI feedback mechanisms for form submissions only*** *— not data display, which is reserved for next week phase 5.*

For the implementation of Phase 4 (Form Submission and Data Insertion), I as a project manager decided to have a **centralized JavaScript file (api.js)** to manage all frontend-to-backend network requests, rather than utilizing multiple page-specific scripts (e.g., register.js, login.js, listing.js).

This architectural decision was made to strictly adhere to the DRY (Don't Repeat Yourself) principle. By centralizing our fetch() logic, we established a single source for our backend endpoints and base URL configuration. This approach not only ensures consistent error handling and JSON parsing across the entire system but also significantly improves maintainability. As the application scales to include more interactive forms in the future, this modular structure will prevent code duplication and streamline future debugging.

II. API Endpoints Implemented

The table below lists all endpoints handled in *api.js*

Method	Endpoint	Notes
POST	/api/auth/register	Standalone form. Inserts user data into MongoDB, returns JWT token.

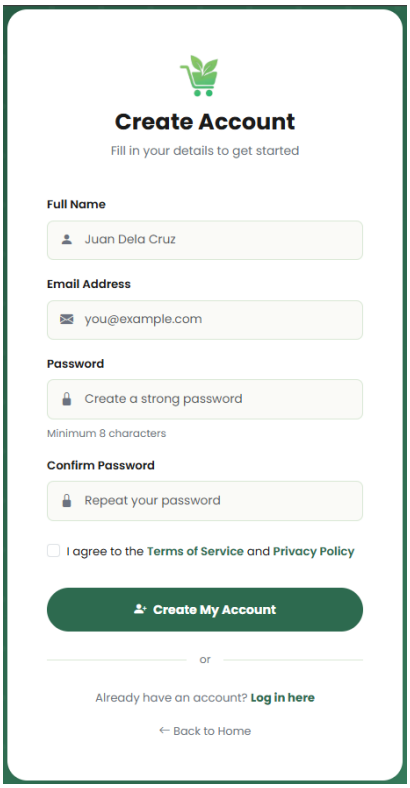
POST	/api/auth/login	Validates credentials, returns JWT. Token saved to localStorage.
POST	/api/products	Requires token from login. Submits as FormData to support image upload via Cloudinary.
PUT	/api/users/:id	Requires token. Updates name/email in MongoDB and syncs localStorage user object.
POST	/api/cart/add	addToCart() function is fully implemented. But cannot be triggered from UI until products are displayed in Phase 5.
POST	/api/orders	checkoutForm handler is complete. But requires cart items in localStorage which are only populated after addToCart() is called in Phase 5 implemetation.

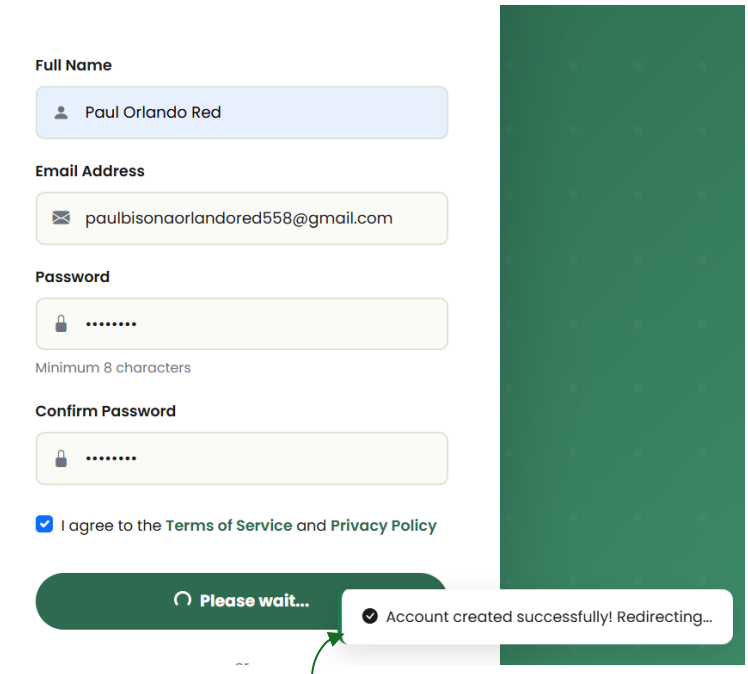
II. HTML Form Interface Screenshots

Screenshots below show each form page open in the browser via Live Server, demonstrating the UI structure that api.js connects to.

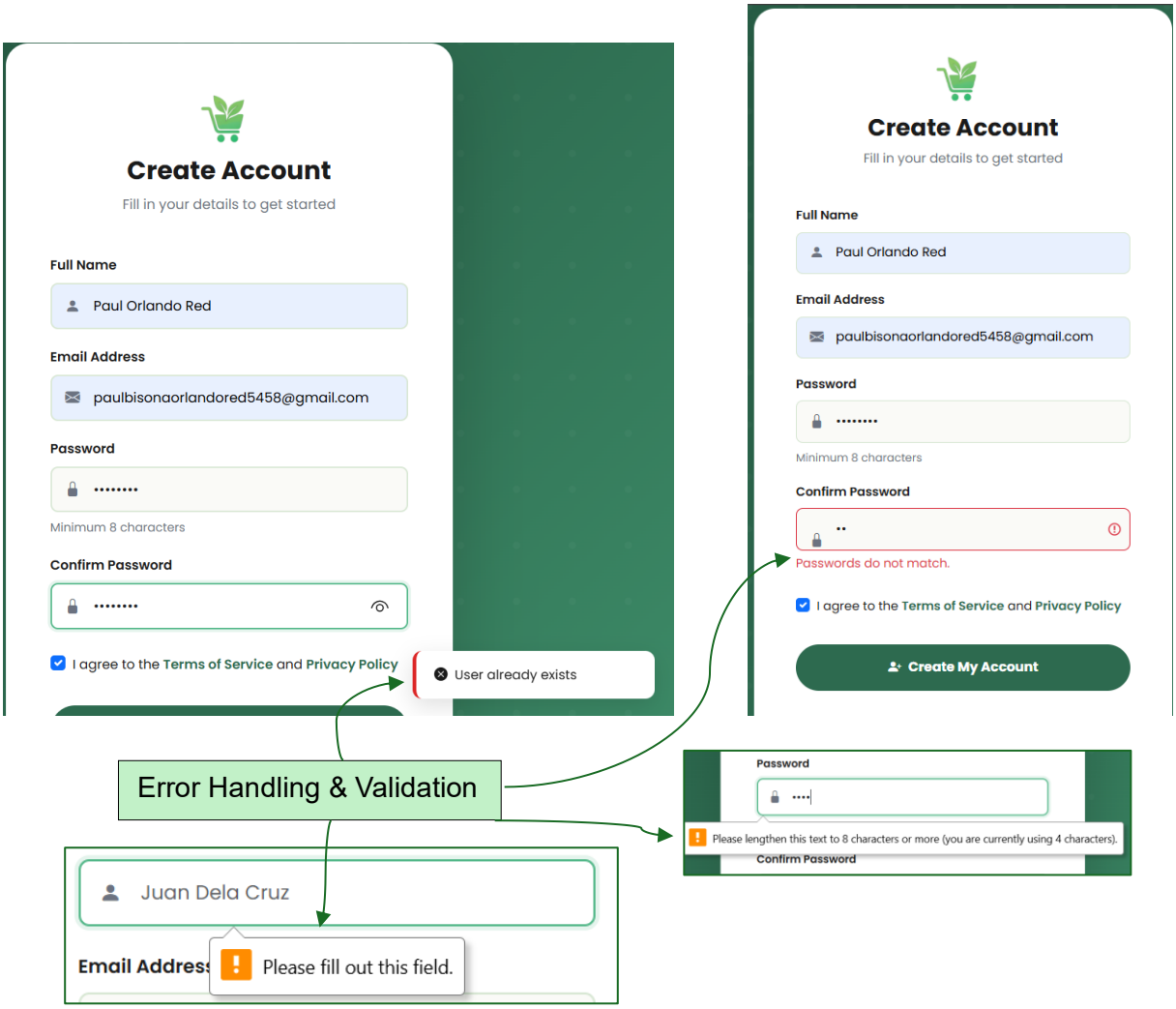
A. Register Form — register.html

Route: POST /api/auth/register





User feedback after successful registration

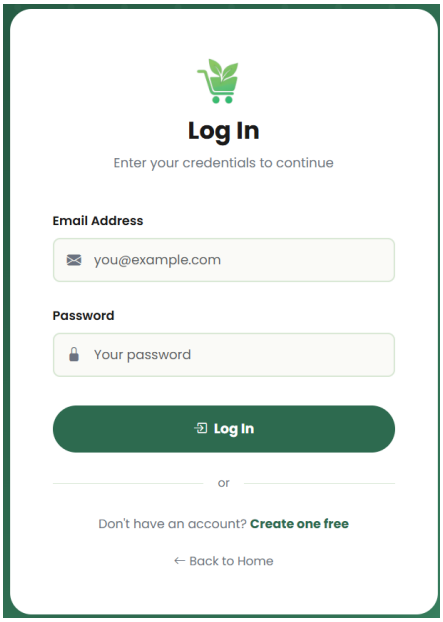


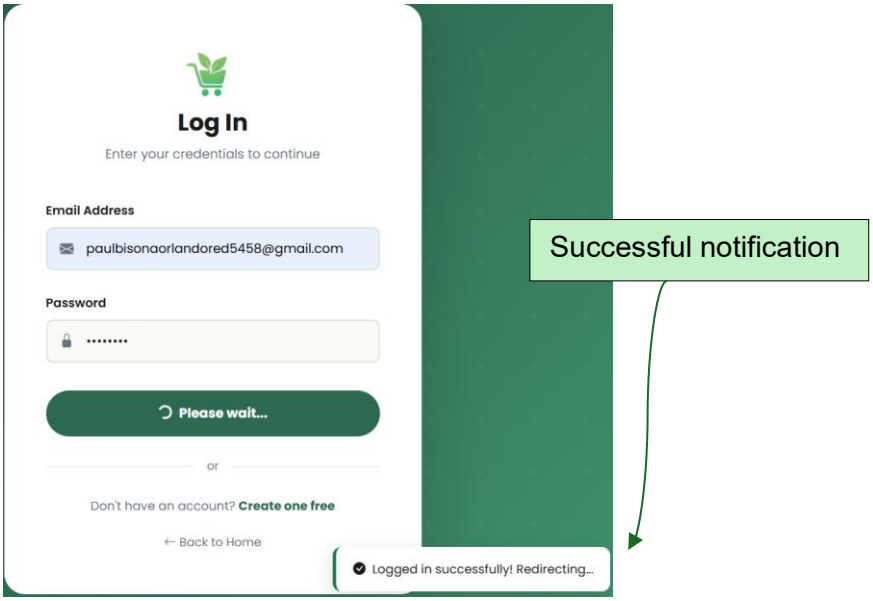
Inserted MongoDB document

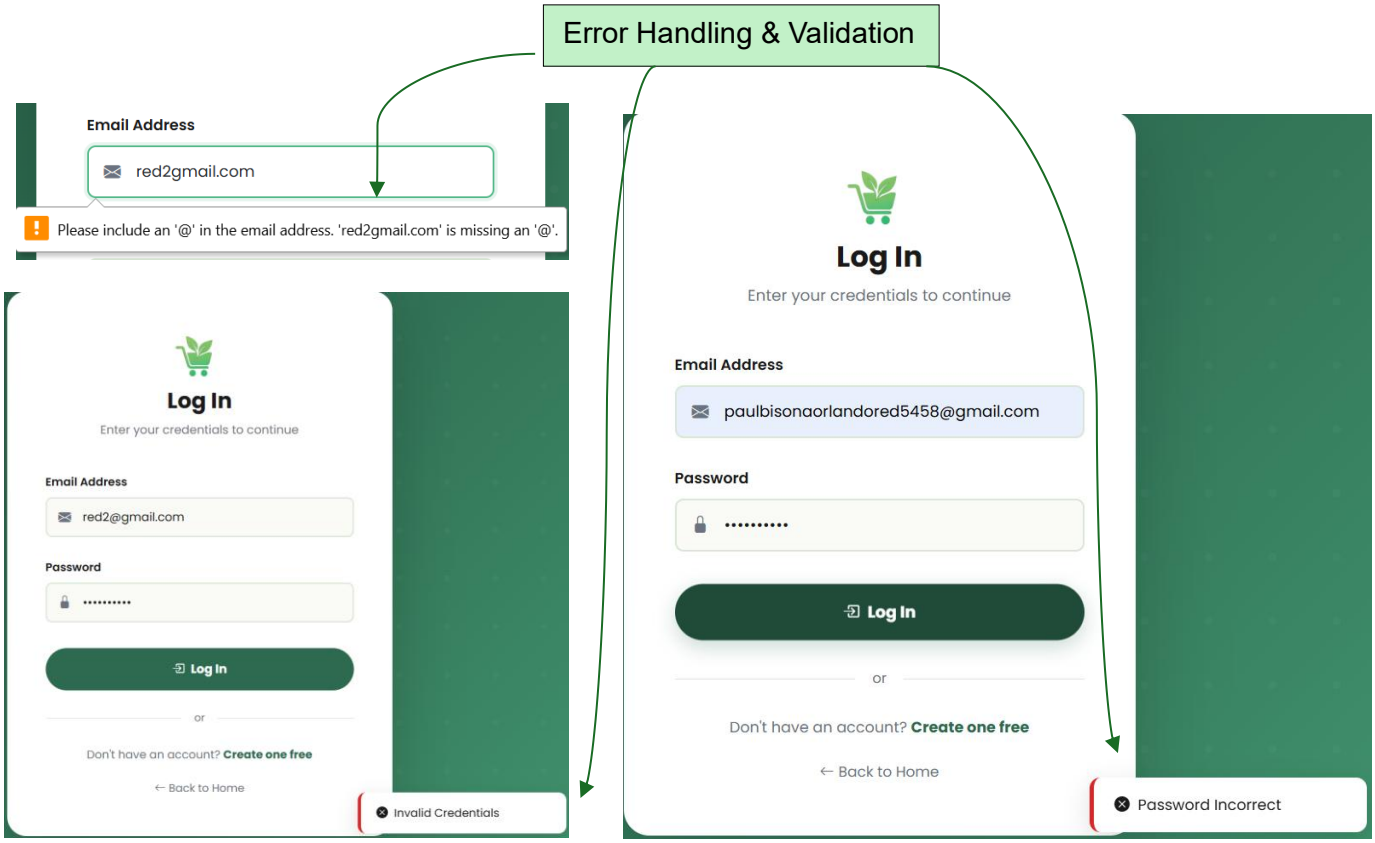
```
_id: ObjectId('69e335ce6a0bacb66a941f1e')
name : "Paul Orlando Red"
email : "paulbisonaorlandored5458@gmail.com"
password : "$2b$10$8XJt1L28VyjtWpY7toajIuZfOY7mduLc1BLoKuHPcI6MwRTU1lAcu"
created_at : 2026-04-18T07:42:06.105+00:00
__v : 0
```

B. Login Form — login.html

Route: POST /api/auth/login



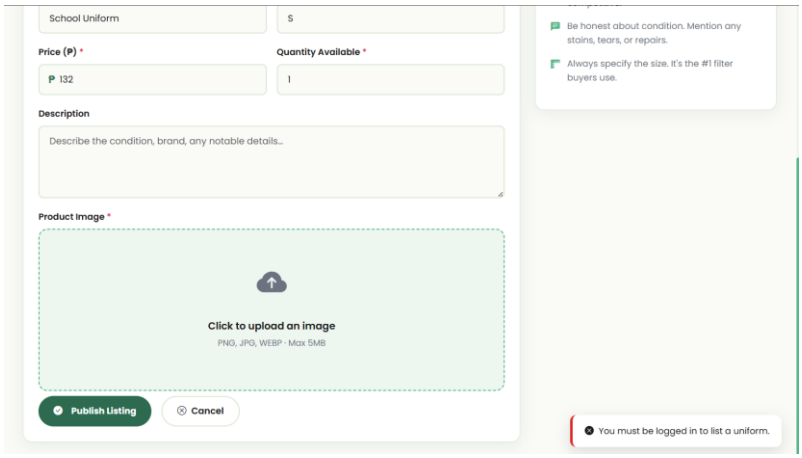


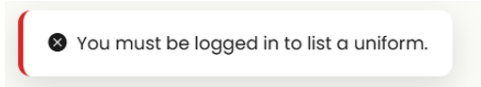


C. Add Listing Form — add-listing.html

Route: POST /api/product

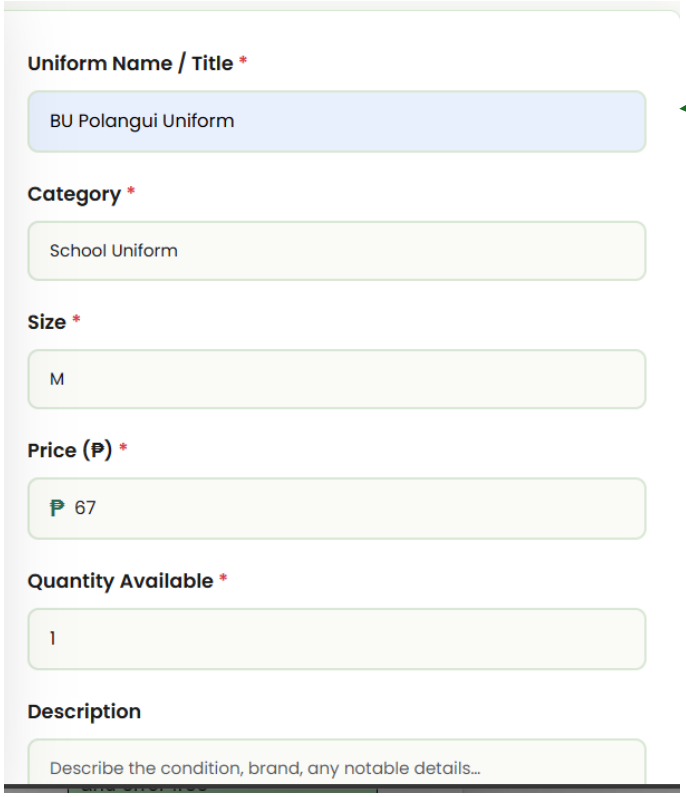
User must be logged in before listing a uniform and will be redirected to login page

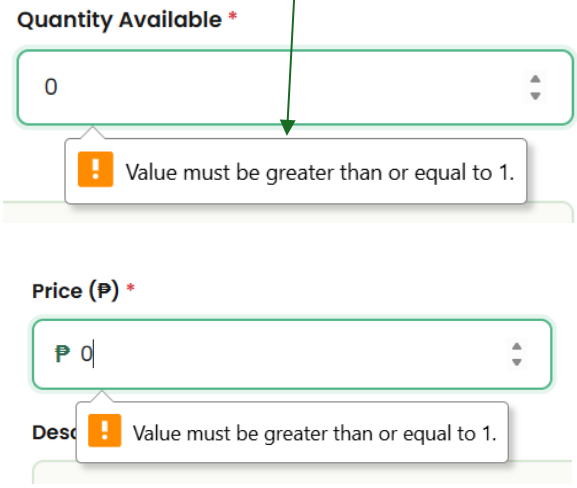




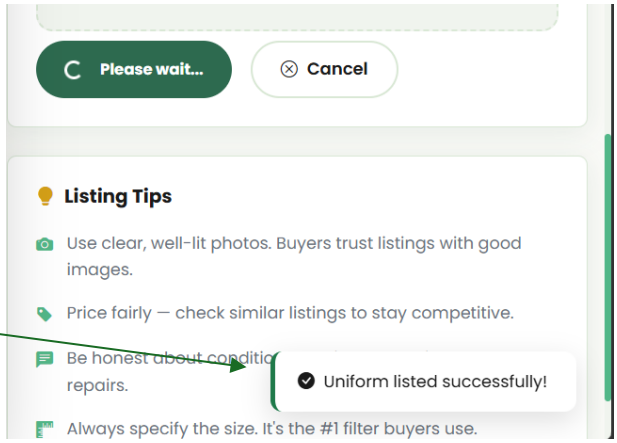
Validation: All fields with asterisks (*) are required to be filled out before publishing

Validate that the Price field and quantity must be atleast 1 or more





Successful listed notification if the user data inputs are valid and error free

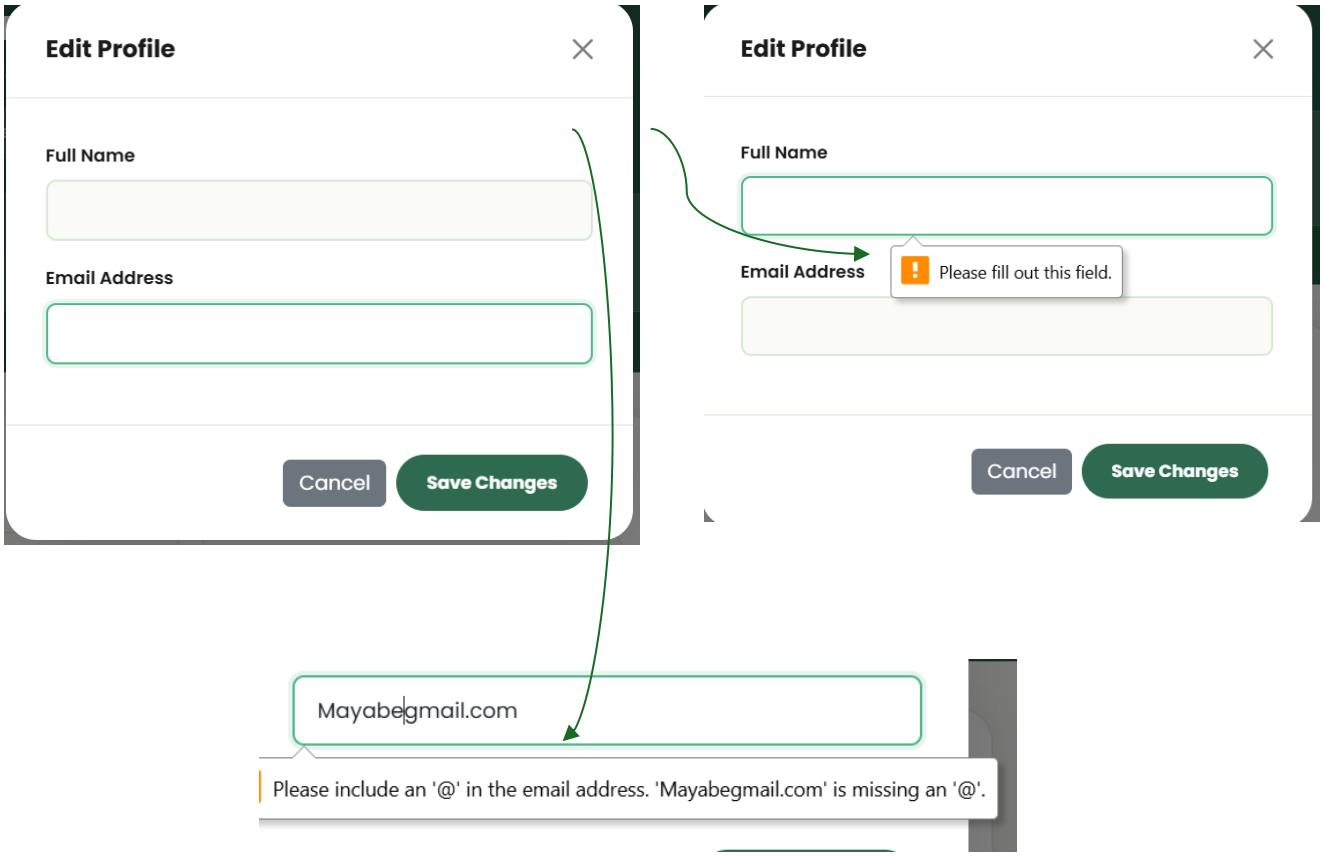


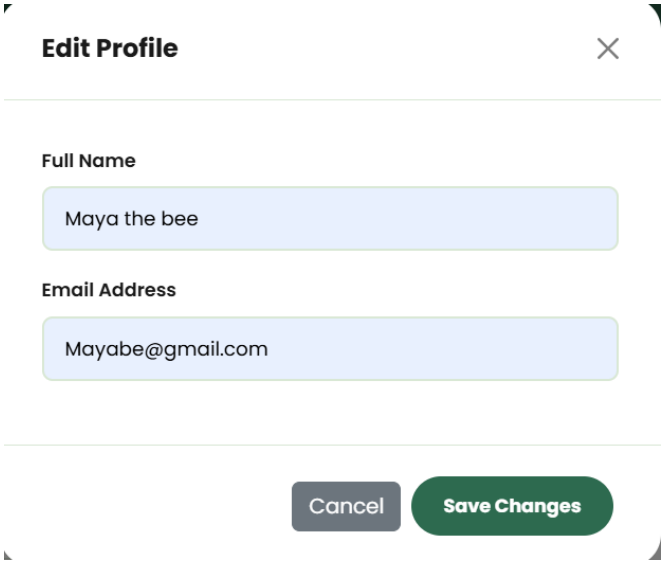
Inserted MongoDB document

```
_id: ObjectId('69e3596301ef5901095ea737')
name : "BU Polangui Uniform"
size : "M"
quantity : 1
price : 67
description : ""
image_url : ""
seller_id : ObjectId('69e335ce6a0bacb66a941f1e')
category_id : ObjectId('69e040e0b6844662221fe951')
created_at : 2026-04-18T10:13:55.176+00:00
__v : 0
```

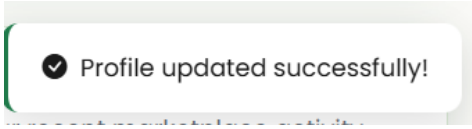
D. Edit Profile Form — profile.html

Route: PUT /api/users/:id





After correct inputs



Edited MongoDB document


```
_id: ObjectId('69e335ce6a0bacb66a941f1e')
name : "Maya the bee"
email : "Mayabe@gmail.com"
password : "$2b$10$8XJt1L28VyjtWpY7toajIuZfOY7mduLc1BLoKuHPcI6MwRTU1lAcu"
created_at : 2026-04-18T07:42:06.105+00:00
__v : 0
```

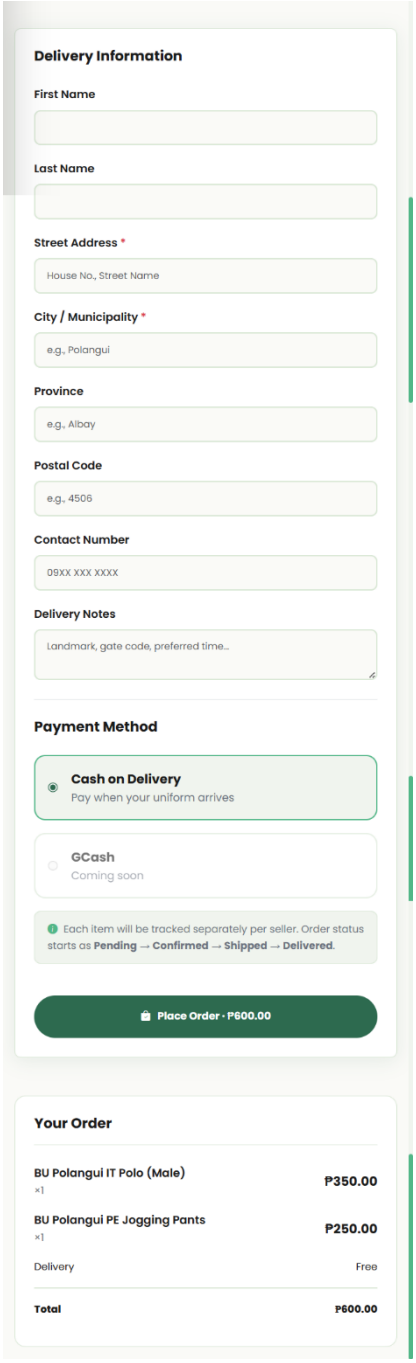

E. Checkout Form — checkout.html

Route: POST /api/orders

Validation: All fields with asterisks (*) are required to be filled out before publishing

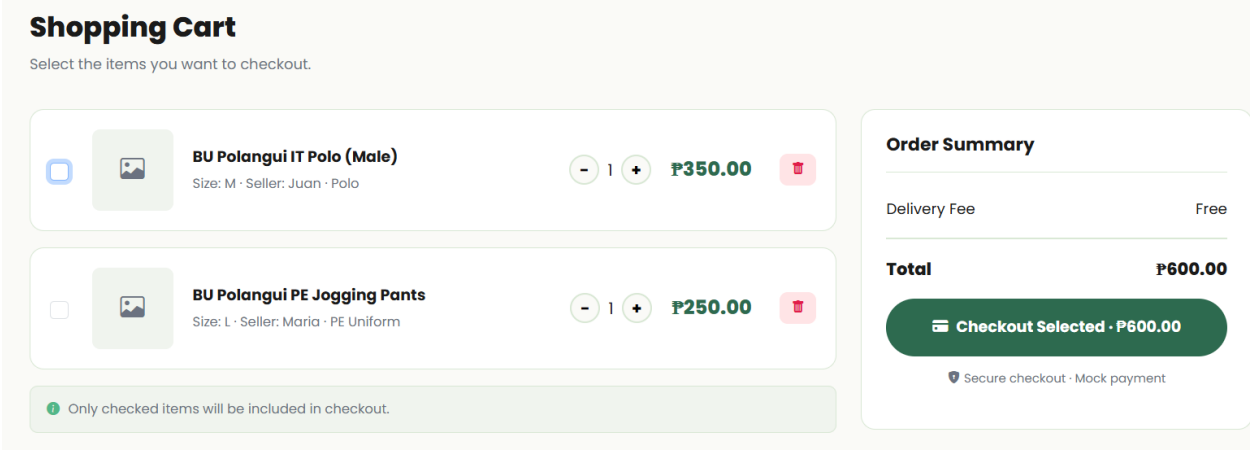
Note:

The checkout form submission handler (POST /api/orders) is fully coded in api.js. However, it depends on cart items being stored in ***localStorage****, which is only populated when the Add to Cart button is clicked on a displayed product. Since product display is a Phase 5 task, the full end-to-end checkout flow cannot be demonstrated via the UI at this stage. The code is in place and ready to function as soon as Phase 5 connects the product display.*



F. Add to Cart

Route: POST /api/cart/add



Note: *The **addToCart()** function is fully written in api.js and handles fetching the user cart, posting the item to /api/cart/add, and mirroring the result to localStorage. It cannot be triggered from the UI because there are no product cards displayed yet to click "Add to Cart" on. This will be wired automatically in Phase 5 when products load from the database onto the dashboard.*

IV. JavaScript API fetch() Connection Code

A. Register Handlers

```
try {
    const res = await fetch(`${API_BASE}/auth/register`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ name, email, password }),
    });
    const data = await res.json();
    if (!res.ok) throw new Error(data.msg || 'Registration failed.');

    saveToken(data.token);
    const meRes = await fetch(`${API_BASE}/auth/me`, { headers: { 'x-auth-token': data.token } });
    const meData = await meRes.json();
    saveUser(meData);

    showToast('Account created successfully! Redirecting...');
    setTimeout(() => { window.location.href = 'dashboard.html'; }, 1500);

} catch (err) {
    setLoading(btn, false, originalText);
    showToast(err.message || 'Something went wrong. Try again.', 'error');
}
```

B. Login Handlers

```
try {
    const res = await fetch(`${API_BASE}/auth/login`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ email, password }),
    });
    const data = await res.json();
    if (!res.ok) throw new Error(data.msg || 'Invalid email or password.');

    saveToken(data.token);
    const meRes = await fetch(`${API_BASE}/auth/me`, { headers: { 'x-auth-token': data.token } });
    const meData = await meRes.json();
    saveUser(meData);

    showToast('Logged in successfully! Redirecting...');
    setTimeout(() => { window.location.href = 'dashboard.html'; }, 1500);

} catch (err) {
    setLoading(btn, false, originalText);
    showToast(err.message || 'Login failed. Check your credentials.', 'error');
}
```

C. Add Listing Handlers

```
try {
    const formData = new FormData(listingForm);

    const res = await fetch(`${API_BASE}/products`, {
        method: 'POST',
        headers: { 'x-auth-token': token },
        body: formData,
    });
    const data = await res.json();
    if (!res.ok) throw new Error(data.error || data.msg || 'Failed to create listing.');

    showToast('Uniform listed successfully!');
    setTimeout(() => { window.location.href = 'my-listings.html'; }, 1500);

} catch (err) {
    setLoading(btn, false, originalText);
    showToast(err.message || 'Something went wrong. Try again.', 'error');
}
```

D. Edit Profile Handler

```
try {
    const res = await fetch(`${API_BASE}/users/${user._id}`, {
        method: 'PUT',
        headers: {
            'Content-Type': 'application/json',
            'x-auth-token': token,
        },
        body: JSON.stringify({
            name: nameInput.value.trim(),
            email: emailInput.value.trim(),
        }),
    });
    const data = await res.json();
    if (!res.ok) throw new Error(data.error || data.msg || 'Failed to update profile.');

    // Update stored user with new values
    saveUser({ ...user, name: nameInput.value.trim(), email: emailInput.value.trim() });

    showToast('Profile updated successfully!');
```

```
    // Close Bootstrap modal
    const modal = bootstrap.Modal.getInstance(document.getElementById('editProfileModal'));
    if (modal) modal.hide();

} catch (err) {
    setLoading(btn, false, originalText);
    showToast(err.message || 'Could not update profile.', 'error');
}
```

D. Checkout & Add to Cart (Pre-built for Phase 5)

Checkout fetch() code

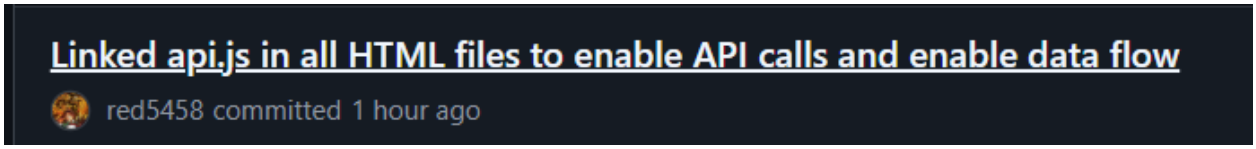
```
try {
    const res = await fetch(`${API_BASE}/orders`, {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json',
            'x-auth-token': token,
        },
        body: JSON.stringify(payload),
    });
    const data = await res.json();
    if (!res.ok) throw new Error(data.error || data.msg || 'Failed to place order.');

    localStorage.removeItem('cart');

    showToast('Order placed successfully!');
    setTimeout(() => { window.location.href = 'my-orders.html'; }, 1500);

} catch (err) {
    setLoading(btn, false, originalText);
    showToast(err.message || 'Something went wrong. Try again.', 'error');
}
```

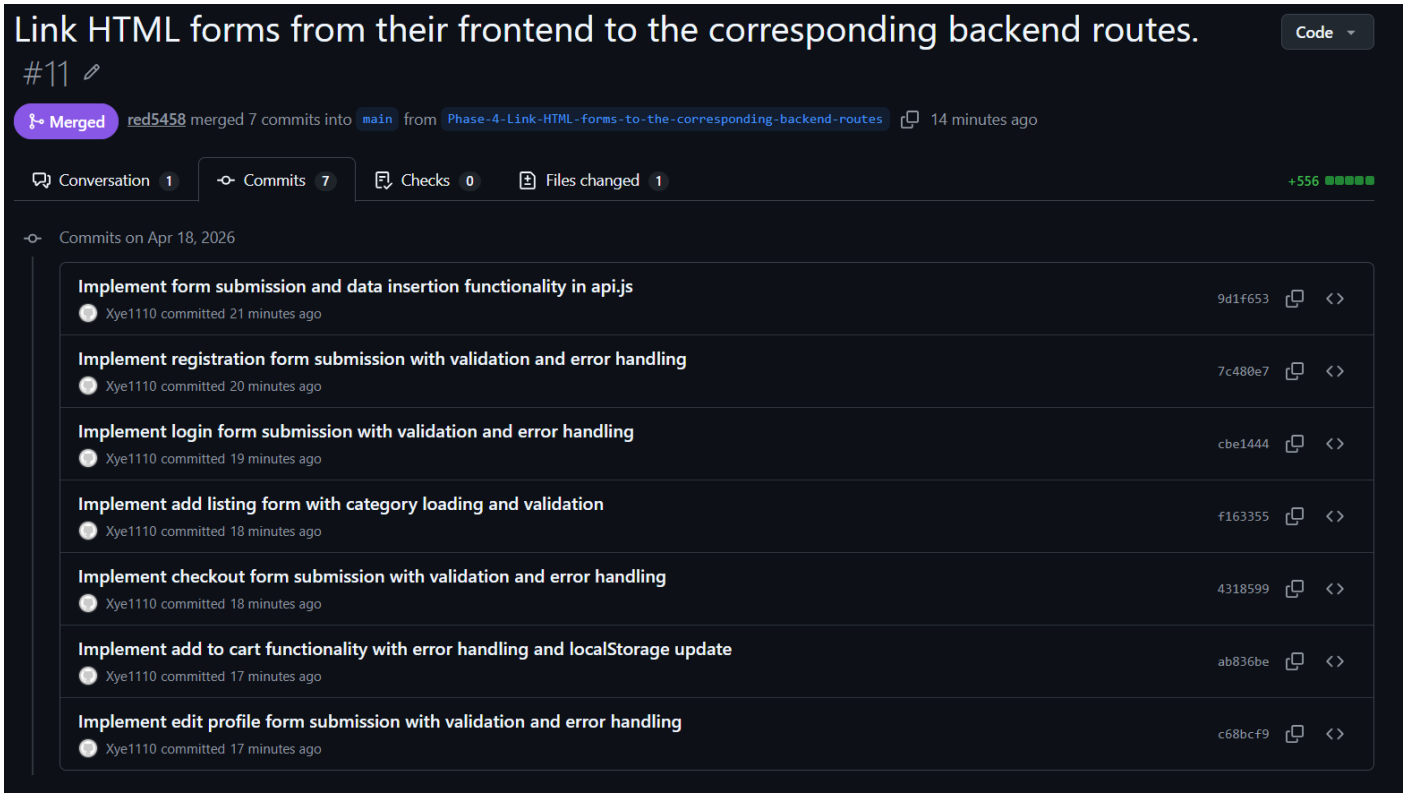
Add to cart fetch() code

```
try {
    // Get or create user cart first
    const cartRes = await fetch(`${API_BASE}/cart/${user._id}`, {
        headers: { 'x-auth-token': token },
    });
    const cartData = await cartRes.json();

    // Add item
    const res = await fetch(`${API_BASE}/cart/add`, {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json',
            'x-auth-token': token,
        },
        body: JSON.stringify({
            cart_id: cartData._id,
            product_id: productId,
            quantity,
        }),
    });
```

```
    });
    const data = await res.json();
    if (!res.ok) throw new Error(data.error || data.msg || 'Failed to add to cart.');

    // Also mirror to localStorage so checkout can read it
    const cart = JSON.parse(localStorage.getItem('cart') || '[]');
    const existing = cart.find(i => i.product_id === productId);
    if (existing) {
        existing.quantity += quantity;
    } else {
        cart.push({ product_id: productId, seller_id: sellerId, price, quantity });
    }
    localStorage.setItem('cart', JSON.stringify(cart));

    showToast('Item added to cart!');

} catch (err) {
    showToast(err.message || 'Could not add to cart.', 'error');
}
```

V. GitHub Commits





V. Short Reflections

Cipriano, Mcxyron B. — Back-End Developer

In this Phase, I was able to see how the backend routes I built in Phase 2 actually work in a real application. Watching the register form send data and return a token that gets reused for protected routes helped me fully understand how JWT authentication works. I helped our frontend developer in fetch() to handle POST request. After it successfully works, I commited and push the changes in the GitHub in a branch. It was satisfying to see everything function correctly through the browser, since it confirmed that what I built on the backend was working as expected. This phase really helped connect the theory with actual implementation.

Hernandez, Kurt Jushua S. — Database Manager / Tester & Debugger

I wasn't assigned in any task in this phase but I worked on testing form submissions and checking the resulting data in MongoDB Atlas and screenshot them. One of the things I learned is how important it is for form field names to exactly match the schema, since even small mismatches can cause issues. This phase made me realize how helpful our earlier schema design was, since it made the integration smoother. Overall, it showed me how everything connects once the frontend starts sending real data.

Registrado, Mharie Franz — Tester & Debugger

I tested all four working forms, including trying **invalid inputs** to make sure validation messages appeared correctly and then reported it to the project manager to coordinate with other developers. This helped me realized how important clear user feedback is, especially with things like inline errors and notifications. This phase gave me a better understanding of how testing works in a partially completed system.

Red, Paul Orlando B. — Project, Database, GitHub manager & Documentation Officer

I managed the GitHub workflow for Phase 4, making sure both frontend and backend changes were properly committed and organized. I also made the documentation and lab report, which helped me clearly understand the full data flow—from form input, to fetch requests, to Express routes, and finally to MongoDB. Even beyond my main role, I stepped in to help review integrations and fix small issues to keep everything working smoothly. This phase gave me a much clearer picture of how a full-stack application works from start to finish.

Romano, Jay L. — Front-End Developer

Building api.js for the frontend was one of the most challenging parts of this phase but I successfully made it with the help of AI tools and from our Backend Developer since I'm not usually familiar in JavaScript but this phase helped me learned how to properly use async/await, send FormData for file uploads, and handle responses without breaking things like headers. I also focused on giving users proper feedback while requests are processing, which made the interface feel more complete. This phase also made me think more about how different parts of the frontend depend on each other, especially for features like cart and checkout.

AI usage: *Around 20% of AI-generated content was used in this lab report to help me, as the Documentation Officer, structure the document and guide what specific screenshots to include along with proper annotations. However, all the information presented in this documentation is based on our actual work, outputs, and understanding of the project. The content reflects our team's efforts, testing, and implementation throughout each phase.*